



Jira Data Center Connector — Credential Provisioning Guide

Version 0.3.0, 2026-08-21

Table of Contents

Overview	1
What kind of account to create	1
Permissions to grant	2
Required, for reading	2
Optional, one per write action	2
Generating the credential	3
Mapping values into Crogl	3
Verifying the setup	3
Reference documentation	4

Overview

The Jira Data Center connector reads issues, projects, and field metadata from a self-managed (on-premises) Jira Data Center instance over its REST v2 API, and—when you choose to allow it—comments on, creates, transitions, and assigns issues. It authenticates with a **Personal Access Token** (PAT) belonging to a Jira user account, sent as an **Authorization: Bearer** header.

These steps are performed by two people:

- A **Jira administrator** (or the intended service user themselves) creates the Personal Access Token and confirms the user's project permissions.
- Whoever administers your Crogl instance enters the resulting values when creating the connector.

You decide how much this connector can do, entirely through the permissions you grant the token's Jira user. Grant browse only, and the connector can read and nothing else—every write attempt fails with a **403** and changes nothing. Grant the write permissions in the next section, and the connector's four write actions (comment, create, transition, assign) become available. There is no delete or bulk-edit capability at all, at any permission level.

Inside Crogl, each write additionally requires the operator to approve that specific action before it is sent—the agent must show exactly what it intends to change and get a yes first. Jira permissions are still your real control, though: anything the token can do, the token can do.

NOTE

This is the **Jira Data Center** (on-prem) connector, not Jira Cloud. If you sign in at a ***.atlassian.net** address, you are on Jira Cloud, which uses a different API version and a different credential (an Atlassian account email plus an API token)—use the Jira Cloud connector instead.

What kind of account to create

Jira Data Center does not have a distinct "service account" object type. A Personal Access Token is minted against an **existing Jira user account**. Best practice is to use a dedicated, low-privilege user (for example **svc-crogl-jira**) rather than a named individual's account, so that:

- the integration keeps working when individuals leave, and
- the token's visibility is scoped to exactly the projects you intend Crogl to read.

The token inherits **that user's** permissions—it can see exactly the projects and issues the user can see, and change exactly what that user could change, and nothing more. Comments and issues the connector creates are attributed to this user, so a recognizable name (**svc-crogl-jira**) also makes the audit trail readable.

IMPORTANT

Personal Access Tokens require **Jira Data Center 8.14 or later**. On older versions the PAT self-service screen does not exist. If your instance predates 8.14, this connector version cannot be used as-is—contact your Crogl FDE, as an HTTP Basic (username + password) variant would be required.

*The exact menu labels below are from Atlassian's published Data Center documentation; if your instance has been customized or heavily version-skewed, the wording may differ slightly. Steps marked **(verify on your instance)** could not be confirmed against a live console for this guide.*

Permissions to grant

All of these are **project** permissions, granted per project through the project's permission scheme. No global administrator permission is required at any level, and none should be granted.

Required, for reading

- **Browse projects**, on every project Crogl should read. Without it, searches return no issues from that project and `get-issue` on a key in that project returns `404`.

Optional, one per write action

Grant only the actions you want the connector to perform. Skipping one is a supported configuration, not a broken setup: the corresponding command fails with a `403` and nothing changes.

Jira permission	What it enables
Add comments	<code>add-comment</code> — post a comment on an issue.
Create issues	<code>create-issue</code> — open a new issue in that project.
Transition issues	<code>transition-issue</code> — move an issue through its workflow. Note that a workflow transition may fire post-functions of its own (notifications, resolution, auto-assignment) that are defined by your workflow, not by Crogl.
Assign issues	<code>assign-issue</code> — set or clear an issue's assignee. To assign work to a particular person, that person also needs Assignable user on the project.

Deliberately **not** needed, and not recommended: **Delete issues**, **Edit issues**, **Manage sprints**, or any administrative permission. The connector has no verb that uses them.

To check or grant these **(verify on your instance)**:

1. As a Jira admin, go to **Administration (gear icon)** → **Projects**, open the target project, and select **Project settings** → **Permissions**.
2. Confirm the token user (directly or via a group/role they belong to) is granted **Browse projects**, plus whichever write permissions above you intend to allow.

TIP A common starting posture is browse-only on every project, plus **Add comments** on the one or two projects your SOC actually works in. You can widen it later without reissuing the token — permissions are evaluated per request, so a change takes effect

immediately.

Generating the credential

Performed while signed in as the token's user:

1. Click the user's **profile avatar** (top-right) and choose **Profile**.
2. In the left-hand menu of the profile page, select **Personal Access Tokens**.
3. Click **Create token**.
4. Give it a recognizable name, e.g. **crogl-connector**.
5. Optionally set an **expiry**. A token with no expiry never rotates on its own; if you set one, calendar a reminder to reissue before it lapses (an expired token surfaces in Crogl as a **401** on the next query).
6. Click **Create**, then **copy the token value immediately** — Jira shows the full token only once. Store it in your secret manager; you will paste it into Crogl in the next section.

Also note your Jira **base URL** (for example **jira.example.com**, or **jira.example.com/jira** if your install uses a context path). You need the host only — no **https://** scheme and no **/rest/api/2** path.

Mapping values into Crogl

One row per **connector.json** credential field:

Crogl field	Where it came from	Notes
host	Your Jira base URL	Bare host, optionally with a context path (e.g. jira.example.com or jira.example.com/jira). No https:// and no /rest/api/2 — Crogl adds the scheme, and the connector adds the API path.
api_token	The Personal Access Token copied above	Secret. Sent as Authorization: Bearer <token> .
skip_tls_verification	n/a (operator choice)	Leave off if Jira presents a certificate from a public CA. Turn on only if your Jira uses a self-signed or internal-CA certificate (common on-prem) and you accept skipping certificate verification.

Verifying the setup

After creating the connector instance in Crogl, run a simple query (e.g. list projects, or **search-issues** with a JQL like **created >= -7d**). Interpret the result:

- **Rows returned** — success.
- **No search-issues results (0 rows)** ... on a valid JQL — the call succeeded and simply matched

nothing. Confirm the token user can **Browse** the project you expected, and that your field/value names match this instance (field identifiers are instance-customizable).

- **401 Unauthorized** — the token is wrong, expired, or revoked. Reissue it and re-enter `api_token`. (Crogl does not validate the token when the connector is created; a bad token first appears here.)
- **403 Forbidden** — the token is valid, but its user lacks the permission that call needs: **Browse projects** if a read failed, or the matching write permission (**Add comments**, **Create issues**, **Transition issues**, **Assign issues**) if a write failed while reads work. Fix the project permission scheme; this is not a Crogl problem. A rejected write changes nothing in Jira.
- **404 Not Found** — most often the `host` value has a stray `https://` or a `/rest/api/2` path baked in, or a wrong context path. Re-check the `host` field is a bare host.
- **TLS / certificate error** — the instance uses a self-signed or internal-CA certificate; enable **Skip TLS verification** on the connector.

If you granted any write permission, verify it once deliberately on a throwaway issue rather than on live work — add a comment to a test issue and confirm it appears in Jira attributed to the service user. A created issue cannot be deleted through this connector, so do the create test in a scratch project.

Reference documentation

The endpoints, request bodies, and permission names in this guide and in the connector's `SKILL.md` were checked against these Atlassian pages, not recalled from training data:

- [Jira Data Center REST API v2 reference \(9.14.0\)](#) — the request bodies and status codes for `POST /issue`, `POST /issue/{key}/comment` (including the `visibility type/value` shape), `POST /issue/{key}/transitions` (204 No Content), `PUT /issue/{key}/assignee` (204), `GET /issue/createmeta`, and `GET /issue/{key}/transitions`.
- [Managing project permissions, Administering Jira applications Data Center](#) — the exact project-permission labels and descriptions used above: **Browse projects**, **Add comments**, **Create issues**, **Transition issues**, **Assign issues**, **Assignable user**.
- [How to set assignee to Unassigned via REST API in Jira](#) — confirms `{"name": null}` unassigns, while `"-1"` selects the project's **default** assignee (which is why this connector sends an explicit null for `--unassign`).
- [Using Personal Access Tokens](#) — the PAT self-service screen and its Jira 8.14+ requirement.